

Writeup: – chest

Informations sur le Crackme

- **Équipe cible** : Non spécifiée
- **Nom du fichier** : writeup_11_chestOK.txt
- **Difficulté estimée** : Non spécifiée
- **Flag découvert** : `ch3st_c0d3_rock`

Résumé

Ce writeup analyse un crackme pour identifier son fonctionnement et retrouver le flag caché.

Outils Utilisés

Coup d'oeil aux chaînes

```
strings -n4 chest | head -n 40
```

Analyse Statique

Désassemblage

```
objdump -d chest > chest.asm
```

Analyse Dynamique

L'exécution du programme a permis d'observer son comportement et d'identifier les points de validation.

Identification du Mécanisme de Validation

LOOP1:

```
mov al, [rsi]
```

```
xor al, 0x42
```

```
sub al, 0x10
```

```
mov [rdi], al
```

```
inc rsi
```

```
inc rdi
```

```
loop LOOP1
```

► Les 8 premiers octets sont décodés par : $((\text{byte} \wedge 0x42) - 0x10) \& 0xFF$.

Puis juste après (0x4010b6) :

```
mov al, [0x4022f6] ; idem pour f7, et f8
```

```
xor al, 0x7
```

```
sub al, 0x2
```

```
mov [0x40237x], al
```

...

Même principe pour octets 8-10.

L'octet 11 est copié brut depuis 0x402255, puis les octets 12-15 subissent à nouveau le combo XOR/-2.

Au final, les 16 octets terminent dans 0x402368–0x402377.

Comparaison avec l'entrée

Découverte du Flag

On repère :

After a shipwreck ...

Bad Password!

Good Job!

On sait déjà qu'il faut trouver le password .

On fait une extraction rapide du flag

petit script Python :

```
data = open('chest', 'rb').read()
va_to_off = lambda va: va - 0x400000
```

```
def b(va):    #lit un octet à l'adresse virtuelle
    return data[va_to_off(va)]
```

```
const = []
```

octets 0-7 :XOR 0x42, -0x10

```
for i in range(8):
    const.append(((b(0x4022ee + i) ^ 0x42) - 0x10) & 0xff)
```

octets 8-10 :XOR 0x07, -0x02

```
for i in range(0x4022f6, 0x4022f9):
    const.append(((b(i) ^ 0x07) - 0x02) & 0xff)
```

octet 11 :copié brut depuis 0x402255

```
const.append(b(0x402255))
```

octets 12-15 :XOR 0x07, -0x02

```
for i in range(0x4022fa, 0x4022fe):
    const.append(((b(i) ^ 0x07) - 0x02) & 0xff)
```

```
flag = bytes(const).decode()
print(flag)
```

Le bon flag trouvé : ch3st_c0d3_rock'

Conclusion

Ce crackme a été résolu en comprenant son mécanisme de validation et en élaborant une stratégie appropriée.

Reconstruction du secret

vvers 0x40108d

Reconstruction du secret

vvers 0x40108d :

```
movabs rsi, 0x4022ee    ; src
movabs rdi, 0x402368    ; dst
mov     ecx, 8
```

Vers 0x40115f puis 0x4011a4

Vers 0x40115f puis 0x4011a4 :

```
lea rsi, [0x4022fe]    ;buffer de lecture (read)
lea rdi, [0x402378]    ;buffer qui contient le secret
mov ecx, 0x10
```

LOOP_CMP

LOOP_CMP:

```
mov al, [rsi]
cmp al, [rdi]
jne Bad_Password
inc rsi
inc rdi
loop LOOP_CMP
jmp Good_Job
```

aucun traitement sur notre saisie : il suffit de saisir exactement ce qui est stocké en 0x402368+